

## **Entrega 3**

**-Juan Sebastián Herrera Corzo 202515767-**

**-Nicolas Arce Sanabria 202513149-**

**-Juan Sebastian Leon 202510399-**

### **Contexto del Problema**

Este sistema cumple el rol de propuesta cuyo objetivo es centralizar y gestionar operaciones internas de un Board Game Café. La solución está diseñada para operar de forma únicamente local dentro del establecimiento, lo que implica que su funcionamiento principal no necesita de conexión a internet. El software soporta la interacción de tres actores fundamentales: los clientes, quienes gestionan sus mesas, pedidos y préstamos, los empleados, que están divididos entre meseros y empleados, pueden consultar y modificar sus turnos y el administrador, quien supervisa el inventario, finanzas y plantilla de empleados. El propósito del sistema es automatizar el control en tiempo real de personas que entran y salen del establecimiento, esto es en pocas palabras el control de aforo, aplicar reglas de compatibilidad en el préstamo de juegos físicos y fijar la facturación de la cafetería, garantizando la persistencia de datos que operan en el negocio.

### **Usos de Herencia y Polimorfismos**

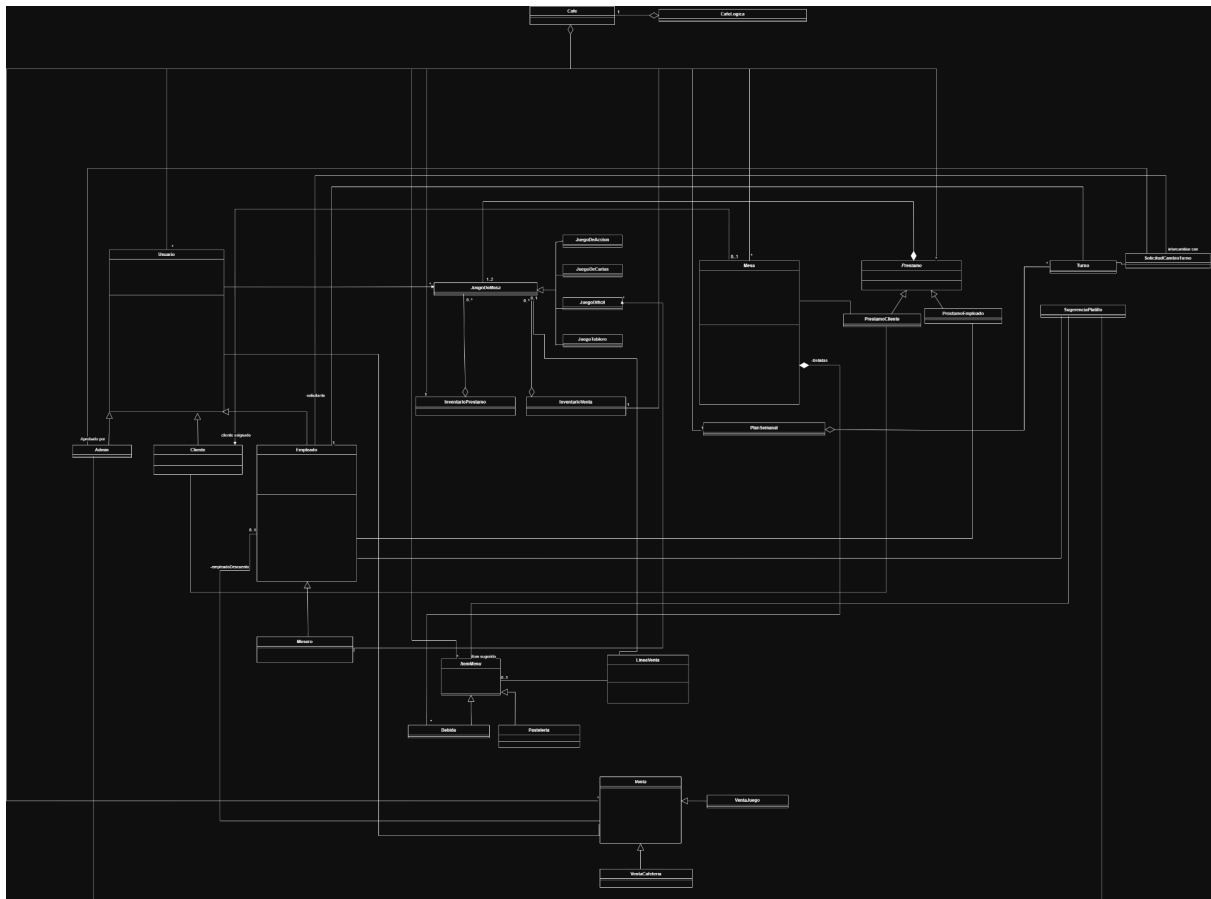
En el diseño de la aplicación aplicamos el patrón de herencia mediante clases abstractas en varios módulos, como usuarios, menú, préstamos y juegos de mesa. Por ejemplo, en lugar de crear clases gigantes y repetir código, lo cual es una mala práctica de programación, creamos la clase abstracta padre Usuario, que contiene datos comunes de login y password. De esta, heredan las clases de Cliente, Admin y Empleado. Esto permite mantener un código más limpio y legible, manteniendo únicamente atributos exclusivos para cada clase correspondiente, como puntos de fidelidad en clientes y Estado en turno en empleados.

Además, la herencia facilitó la implementación del polimorfismo, lo cual fue fundamental para simplificar la lógica de facturación. La muestra más precisa de esto está en la clase abstracta Venta y su método calcularTotal(). Al tener dos tipos de ventas (de juego y cafetería), el sistema no necesita usar múltiples condicionales, sino solamente necesita llamar al método calcularTotal del objeto venta correspondiente. El polimorfismo permite saber al sistemas si debe aplicar 19% de IVA si es juego o el 8% si es cafetería.

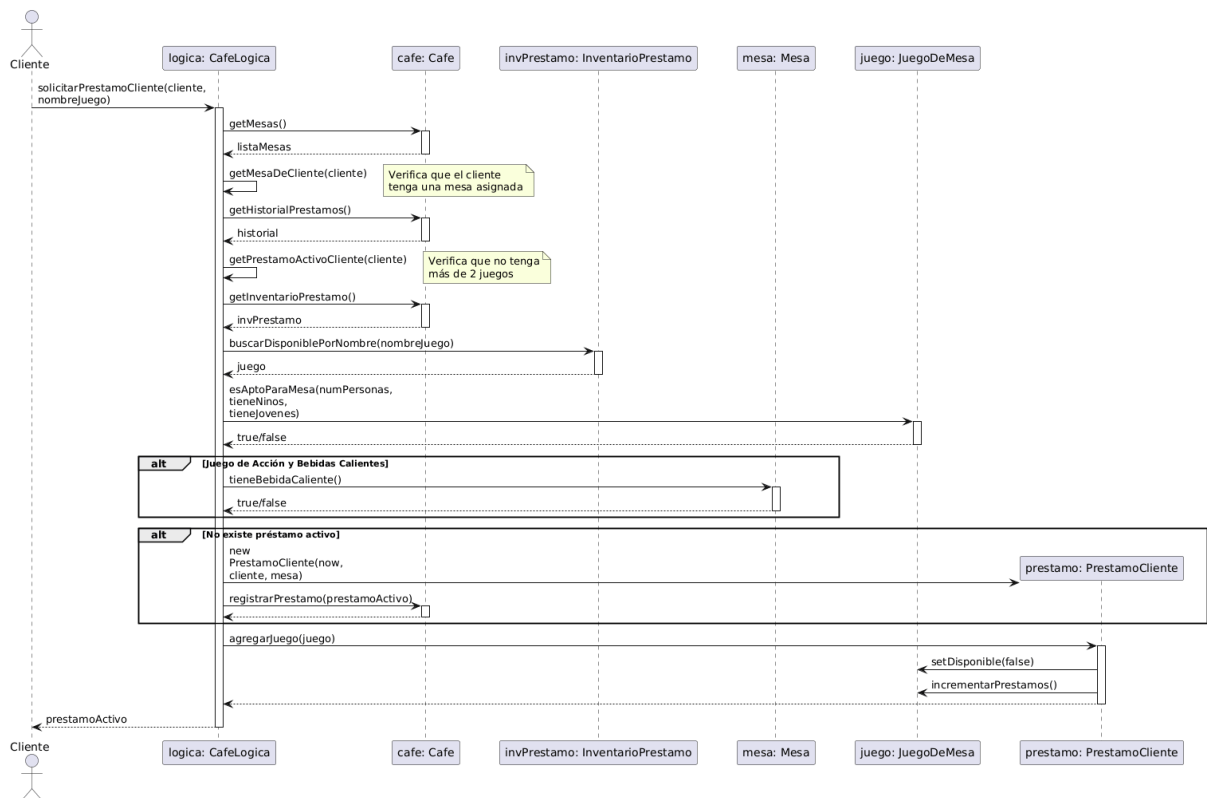
### **Controlador**

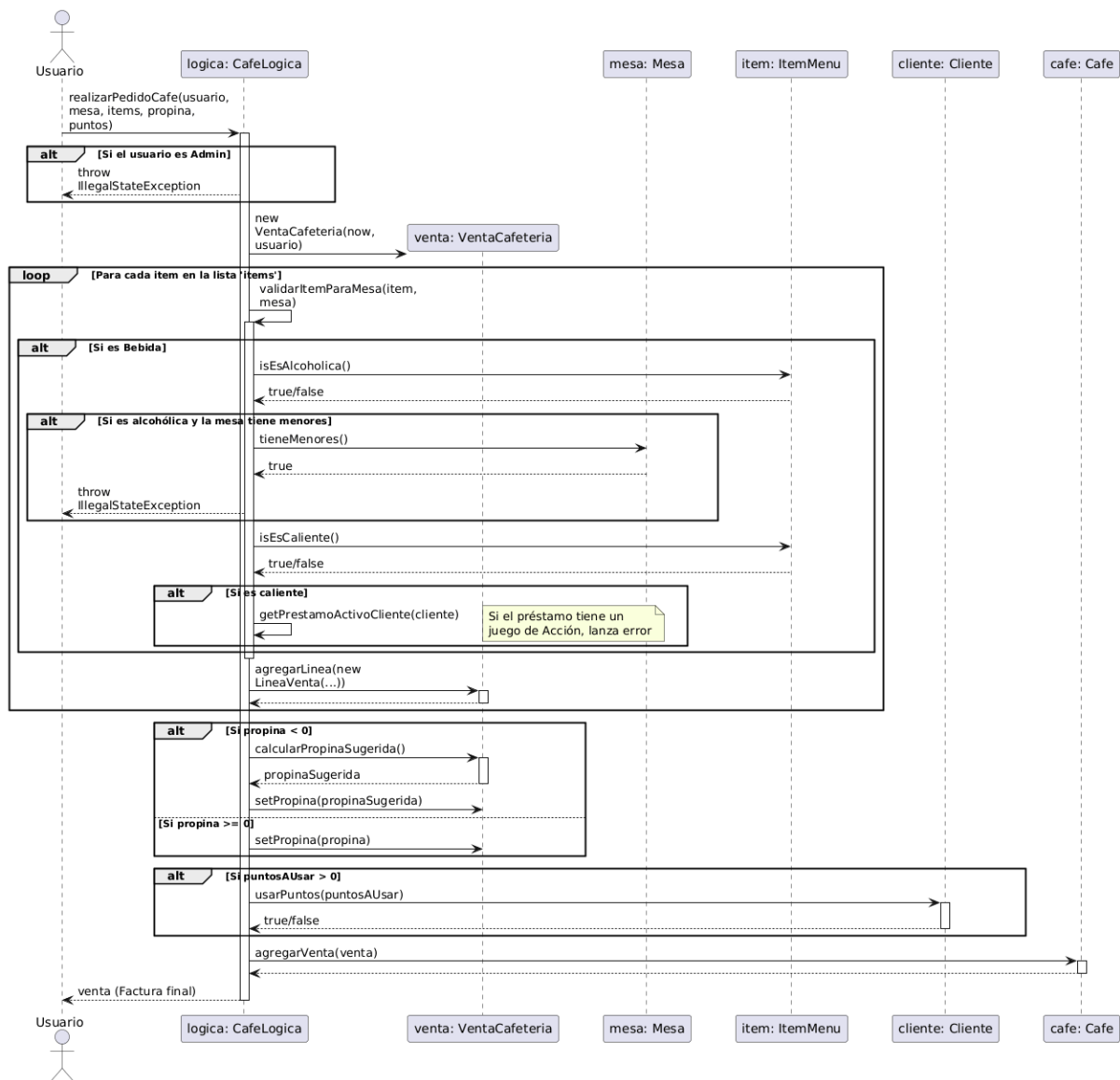
Decidimos separar la lógica del negocio de los datos puros. Por esta razón, la arquitectura del modelo cuenta con la clase Cafe, que actúa únicamente como contenedor de datos (listas de





**Figura 2. Modelo de alto nivel sin métodos ni atributos**





Los diagramas de secuencia nos permiten entender cómo funciona el código de una forma visual y dinámica. Las clases que aparecen en recuadros morados en la parte superior e inferior indican el momento en el que la función utiliza un método particular de esa clase, por ejemplo, new venta Cafetería señala la clase venta. Por otro lado, estas secuencias deben leerse de arriba a abajo, así se puede leer el flujo de la función de manera lógica. Además, los cuadros ALT indican que hay condicionales que validan que la función se pueda ejecutar o no.

## PERSISTENCIA:

Para la parte de persistencia en el guardado de datos de la parte de la clase Café se decidió por una estructura JSON, esto se debe a que su lectura, escritura y verificación de la información es más sencilla en comparación con otros formatos como los binarios, ya que su estructura es más simple. En el proyecto la persistencia está dividida en cinco partes, una de estas es la encargada de llamar y administrar el resto, las cuales están asociadas con un atributo específico del Café.

1. **CentralPersistencias:** Esta clase es la encargada de cargar y salvar los datos de los atributos de Café teniendo en cuenta las demás persistencias relacionadas con algún atributo de Café. Dado que sus principales funciones son cargar y guardar la información, los únicos métodos presentes son los ya mencionados. El método `cargarTodo` requiere para su funcionamiento los distintos nombres de los archivos con la información y el objeto Café el cual es donde se va a importar los datos; en la parte de su funcionamiento, está invoca al resto de las persistencias correspondientes a cada atributo para posteriormente llamar al método correspondiente de la carga de datos. Por otro lado, el método `guardarTodo` cumple con la función de ejecutar el modo guardado de cada atributo llamando el método de su persistencia correspondiente.
2. **PersistenciaMenu:** Esta clase es la encargada del manejo de datos del atributo menú del objeto Café. Esta clase carga y guarda la información del menú la cual es una lista que contiene todos los alimentos y bebidas disponibles en la cafetería. El método `cargarMenu` es el encargado de importar la información de un archivo al objeto Café, este método primero revisa que el archivo que le dieron exista para posteriormente empezar a leer el archivo y poder recorrer toda la lista, y cada vez que va por un elemento identifica si es de tipo pastelería o bebida para crear el objeto y agregarlo al menú de Café. Por otro lado, `salvarMenu` se encarga de guardar los datos en un archivo JSON, este método primero crea un array de JSON donde se va a guardar la información del menú, para esto se recorre todo el menú convirtiendo cada elemento en un objeto JSON guardando sus atributos dependiendo si es una bebida o parte de la pastelería para finalmente crear el archivo.
3. **PersistenciaMesa:** Esta clase es la encargada de cargar y guardar la información relacionado con el atributo mesas de Café. Este atributo contiene la lista de mesas disponibles en la cafetería con sus correspondientes características. Como ya se había mencionado el primer método es `cargarMesas`, el cual consiste en importar la información de mesas desde un archivo. Primero verifica si el archivo existe; luego lo lee para posteriormente recorrerlo mirando en cada elemento las características que contiene para crear la mesa y agregarla a Café. En cambio el método `salvarMesas` se encarga de exportar la información de las mesas a un archivo JSON. Primero crea un arreglo JSON donde se almacenará la información. Luego recorre la lista de mesas del objeto Café y convierte cada una en un objeto JSON con sus respectivos atributos. Finalmente, genera y guarda el archivo.
4. **PersistenciaUsuario:** Esta clase es la encargada de gestionar la información acerca de los usuarios registrados en el Café sin importar si son: clientes, empleados o administradores; conserva la misma información. El primer método es el de carga de los datos desde un archivo. Al inicio verifica la existencia del archivo para empezar a leerlo y recorrerlo dato por dato. En cada uno de los recorridos este recolecta la información de cada uno de los usuarios, y dependiendo de estas los va agregando al café dependiendo el tipo específico que sea. El otro método es el de salvar los usuarios de una cafetería a un archivo. Primero inicia un arreglo de tipo JSON donde guardar los datos y empieza a recorrer la lista guardando las

características de los usuarios en formato json para poder agregarlos al arreglo y finaliza creando el archivo correspondiente.

5. `PersistenciaInventarioVenta` - `PersistenciaInventarioPrestamo`: Estas clases aunque distintas poseen características muy similares debido a que la estructura principal es prácticamente la misma. Ambas guardan y cargan la información de una lista de juegos, mientras que en uno se encuentran los que están a la venta al público, en el otro se encuentran los que pueden ser prestados. El primer método en común es el de carga, primero se verifica si el archivo existe, para leerlo y recorrerlo para recolectar la información de cada juego. Ya teniendo las características específicas del juego se crea con el método `crearJuego` para agregarlo ya sea en el inventario de venta o préstamo. El método `crearJuego` se encarga de generar el juego a partir de la información suministrada. Según el tipo de juego, crea la instancia correspondiente, ya sea de acción, cartas, de tablero o si es difícil. Por último, el método de salvar es el encargado de exportar los datos. Al inicio este crea un arreglo donde se va a guardar la información, posteriormente se recorre la lista ya sea de venta o préstamo para recolectar las características de cada juego, convertirlos al formato JSON y guardar el juego en el arreglo para cuando se complete el recorrido crear el archivo y guardarlo en el lugar correspondiente.
6. `PersistenciaTorneo`: Esta clase es la encargada del manejo de datos del atributo torneos de Café conservando todas las características. Primero se tiene el método de carga de los torneos, donde se verifica la existencia del archivo para poder leerlo después de la confirmación y recorrerlo. Posteriormente, recolecta la información de cada uno y verifica que el juego del que consiste el torneo se encuentre en la cafetería para crear el torneo dependiendo el tipo que sea con su correspondiente cupos y mapas de usuarios. Al final tenemos el método de salvar la información del Café a un archivo, donde primero se crea el arreglo JSON donde se van a albergar los torneos. Después, recolecta la información de cada torneo con su tipo, para la parte de los participantes se crea otro arreglo el cual contiene todos los usuarios inscritos a la competencia. Al finalizar el recorrido crea el archivo y lo guarda en la carpeta correspondiente.
7. `PersistenciaTurnos`: Esta clase es la encargada de manejar los datos relacionados con los turnos en la cafetería. Primero que nada se tiene el método de cargar los turnos, los cuales se encuentran en un archivo JSON. Al inicio se verifica que el archivo exista para poder empezar a recorrerlo. A la hora de recorrerlo debe reconocer si hace parte de las solicitudes a cambio de turno o ya es el turno definido por el plan semanal. Entonces al identificar cada una, recolecta sus características puntuales para ir creando el elemento y agregarlo al café. Por último tenemos el método de salvar los turnos, este consiste en exportar los datos a otra carpeta. Al inicio crea un arreglo donde va a guardar el plan semanal ya definido del café, para después recorrerlo e ir recolectando cada uno de los turnos para convertirlos en un objeto JSON. Además, crea otro arreglo el cual va a contener la información acerca de las solicitudes de cambio de turno que aún están pendientes. Para este caso, recorre la lista recopilando la información de cada elemento para al final agregarlo al arreglo en formato JSON. Al finalizar ambos recorridos procede a crear el archivo para posteriormente guardarlo.

## HISTORIAS DE USUARIOS

1. **Administrador:**  
Como administrador del café, quiero poder organizar y verificar el estado del café para su perfecto funcionamiento. Para poder cumplir con esto soy el encargado de guiar a los

empleados para la correcta atención hacia los clientes, además de manejar los aspectos administrativos del local.

- Registro de Empleados: Como administrador, quiero poder registrar empleados en el sistema para que puedan acceder a sus funciones dentro de la aplicación.  
Entradas: El nombre de usuario con su respectiva contraseña y trabajo en el que se va a desempeñar .  
Salidas: Retorna el usuario creado con sus datos; error en caso en el que no agrega los datos necesarios o faltan algunos.
- Creación de Torneos: Como administrador, quiero crear torneos (Amistosos o Competitivos) para aumentar el flujo de clientes en el café.  
Entradas: Nombre del torneo, juego, fecha, límite de participantes, tipo (Amistoso/Competitivo), tarifa de entrada (solo si es Competitivo).  
Salidas: Mensaje de éxito y torneo visible en el catálogo; error si faltan datos obligatorios.
- Gestión de Turnos: Como administrador, quiero asignar turnos a los empleados y revisar solicitudes de cambio para mantener el café operando correctamente.  
Entradas: Para la creación de turnos es necesario el empleado y la hora la cual se le va asignar. Para la parte de solicitudes se decide si aceptarlas o no, para esto es necesario los empleados involucrados y el turno al cual afecta.  
Salidas: Al final se genera el turno correspondiente a la hora de crearlo, si se quiere eliminar se tiene en cuenta las condiciones de la cafetería para saber si eliminarlo o no. Por parte de la solicitudes al rechazarla se muestra el estado de la solicitud en rechazado, en cambio si se acepta se muestra el estado a aceptada y se genera el trámite para el cambio de turno.
- Gestión del menú: Como administrador, quiero verificar y actualizar los elementos del menú (pastelería y bebidas), y revisar sugerencias de los empleados.  
Entradas: Se necesita una descripción del ítem, con su mismo ítem creado por el empleado correspondiente.  
Salidas: Se agrega a la lista donde se encuentran el resto de ítems si se acepta la solicitud, en cambio, si se rechaza no se agrega.
- Acceso a las Finanzas: Como administrador, busco conocer el estado financiero de la cafetería con las cuentas precisas de las ganancias, impuestos, propinas o costos.  
Entradas: Dependiendo el tiempo del cual quiero filtrar la información y saber la información se selecciona el tiempo con un intervalo o día preciso  
Salidas: Una cifra la cual representa las ganancias, propinas o costos.

## 2. Empleado:

Como empleado del café, quiero poder ayudar con las tareas del café a la hora de atender a los clientes para que el café siga funcionando de la mejor manera.

- Consulta de Turno: Como empleado, quiero ver mi turno semanal para organizarme y, si es necesario, solicitar un cambio.  
Entradas: Para pedir cambio necesito a otro empleado con el cual intercambiar, el turno del cual voy a pedir el cambio y mi mismo usuario.  
Salidas: Crea la solicitud correspondiente para que sea aprobada, o arroja error ya sea por información mal dada o por falta de empleados.
- Sugerencia de Platillos: Como empleado, quiero sugerir nuevos platillos o bebidas para enriquecer el menú del café.

Entradas: Dar descripción con su ítem correspondiente.

Salidas: Se agrega a las solicitudes a procesar por el administrador.

- Proceso de Venta: Como empleado, quiero registrar ventas de comida y juegos para llevar el control de caja del café.

Entradas: Se necesita el inventario de juegos a vender o el menú del café dependiendo del caso, el usuario, código de descuento o bono si se requiere y puntos a usar si el usuario lo quiere .

Salidas: Al final se regresa todo lo relacionado con la venta si fue exitosa, sin embargo, arroja error ya sea por errores en el ingreso de la información o comprobación de la misma como puntos insuficientes, código o bono invalido.

- Gestión de Préstamos: Como empleado, quiero registrar y administrar los préstamos de juegos de mesa para llevar un control ordenado.

Entradas: A la hora de prestarlo se necesita la información del juego solicitado, en cambio para regresarlo se necesita la misma información, pero con la mesa en la que se encuentra.

Salidas: Se crea el préstamo correctamente o genera error ya sea porque ya está en uso, no se encuentra en inventario o se entregó mal la información. .

### 3. Cliente:

Como cliente del café, quiero poder hacer uso del servicio que me ofrecen, ya sea por parte de los juegos de mesa o de la comida para llevar una buena experiencia y seguir disfrutando del café.

- Registro en el Sistema: Como cliente nuevo, quiero registrarme en la aplicación para acceder a los servicios del café.

Entradas: Se necesita el login el cual va a ser el nombre de usuario, además, de la contraseña.

Salidas: Se crea el usuario con la información suministrada, pero arroja error si el login ya está en uso por otro usuario.

- Inscripción a Torneos: Como cliente, quiero inscribirme a un torneo tomando hasta 3 cupos para participar con mis amigos.

Entradas: Se dan mis datos como jugador, el torneo al cual aspiro participar con los cupos los cuales compré.

Salidas: Si se acepta la inscripción se muestran los cupos comprados, en cambio, suelta error si la información no es la adecuada, ya sea el número de cupos o no hay cupos suficientes, entre otros.

- Sistema de Puntos de Lealtad: Como cliente frecuente, quiero acumular y canjear puntos por mis compras para obtener beneficios por mi fidelidad.

Entradas: Necesito el monto de la venta realizada con mi usuario.

Salidas: Se agregan los puntos a mi usuario dependiendo la información suministrada.

- Registro de Juegos Favoritos: Como cliente, quiero guardar mis juegos favoritos para recibir un servicio más personalizado en futuras visitas.

Entradas: Digo cual es el juego del cuál me gusta para que lo tengan en cuenta en mi futuro con mi usuario.

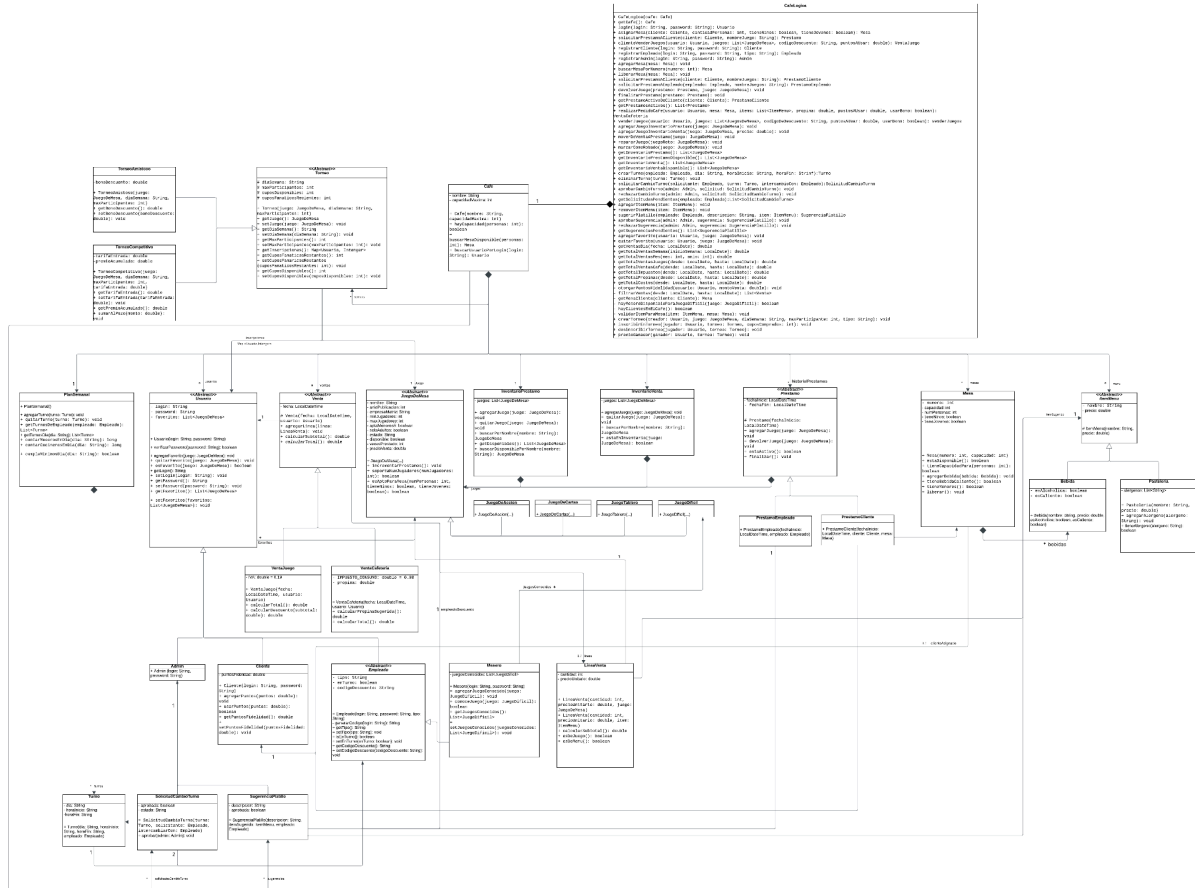
Salidas: Se agrega a mi usuario para un servicio personalizado en un futuro.

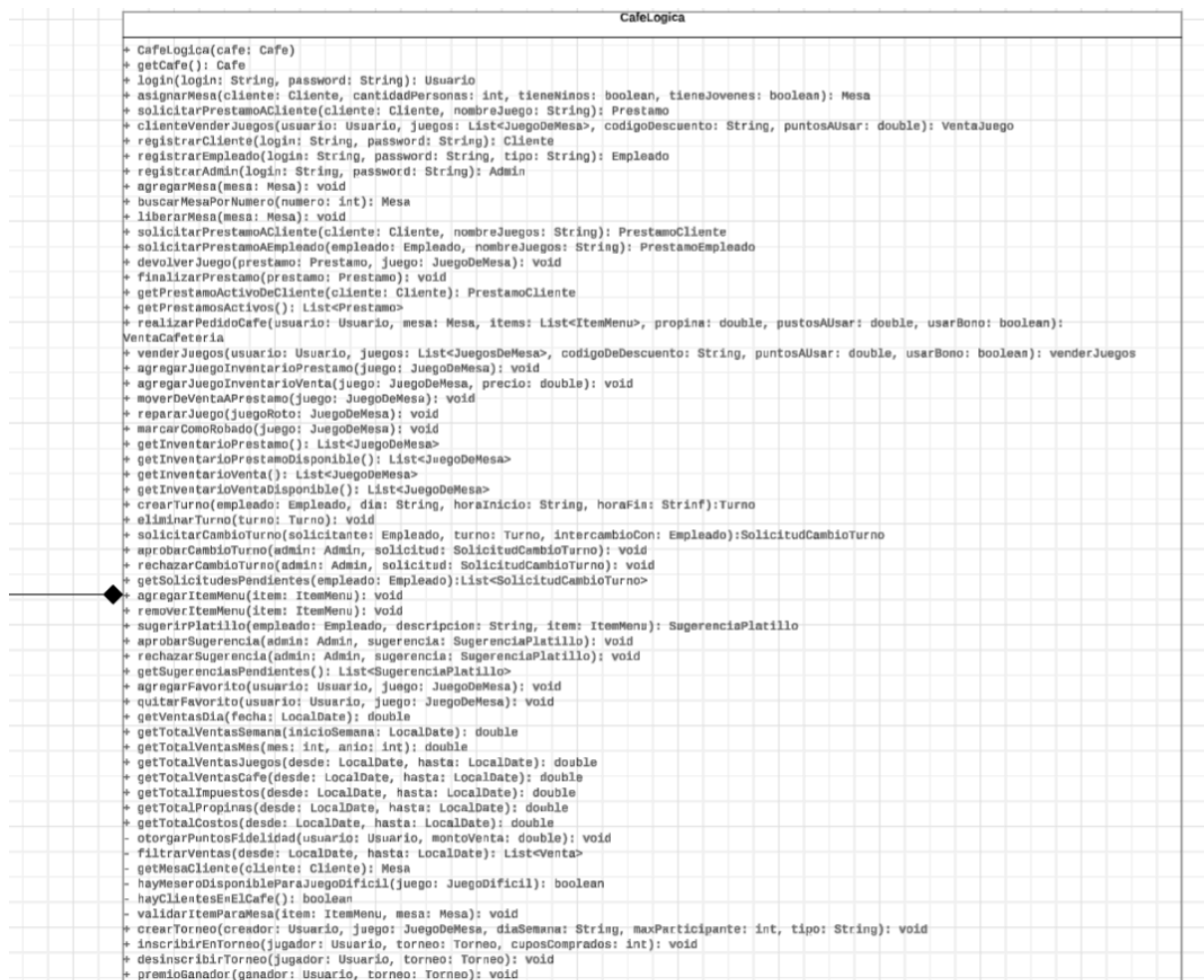
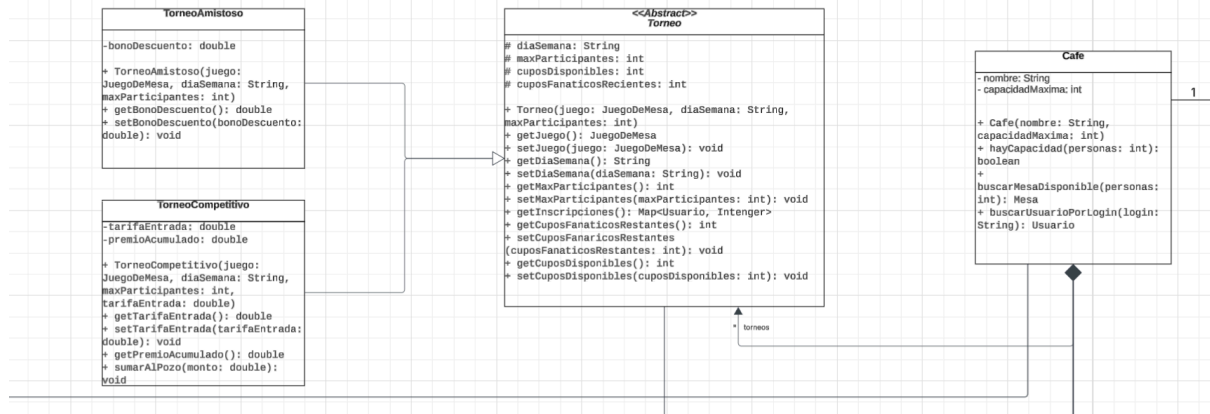
- Compra de Juegos: Como cliente, quiero poder comprar juegos que me interesen directamente en el café.

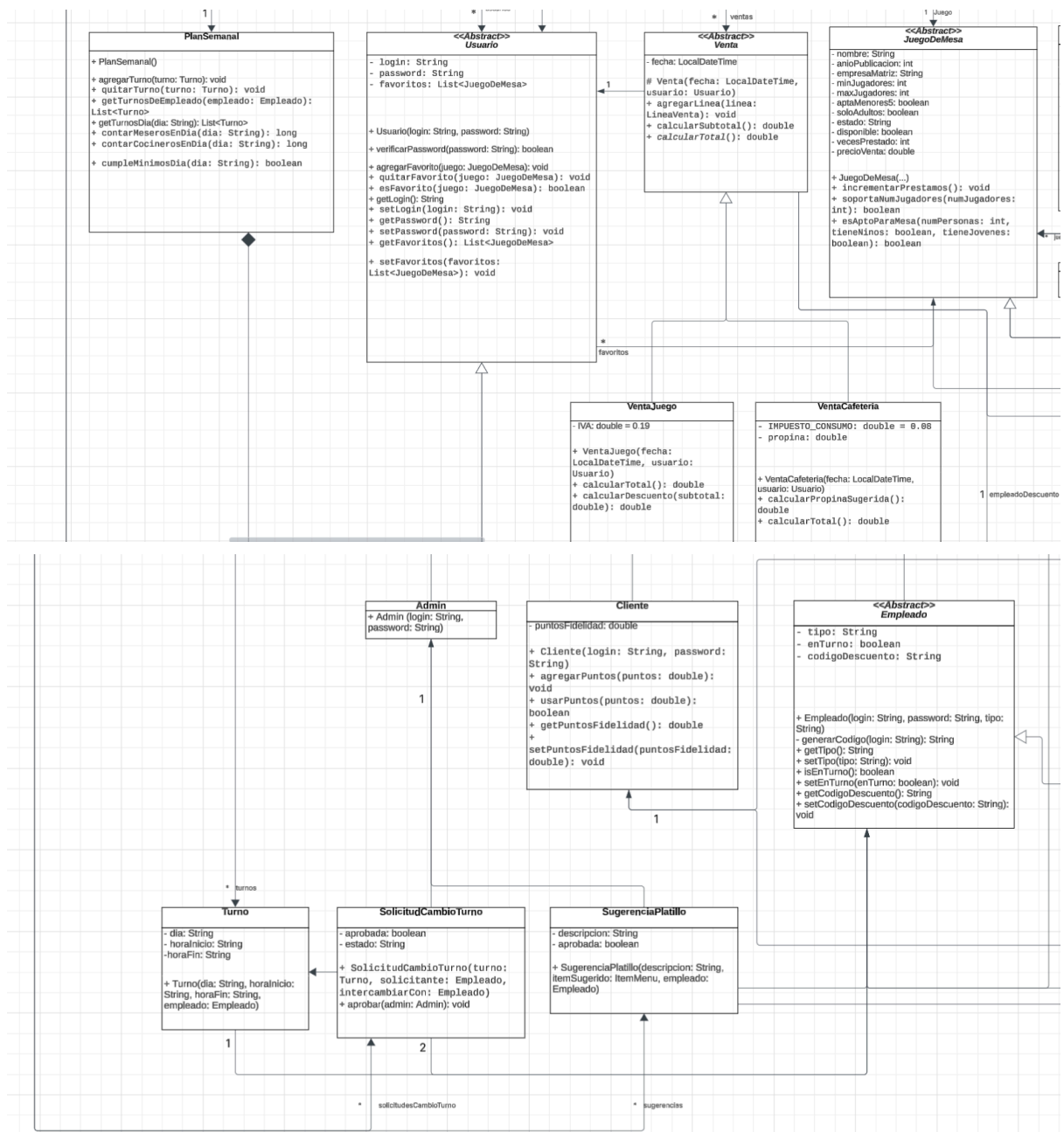
Entradas: Selecciono el juego a comprar, doy mi usuario, y si tengo puntos, bonos o descuento se agregan.

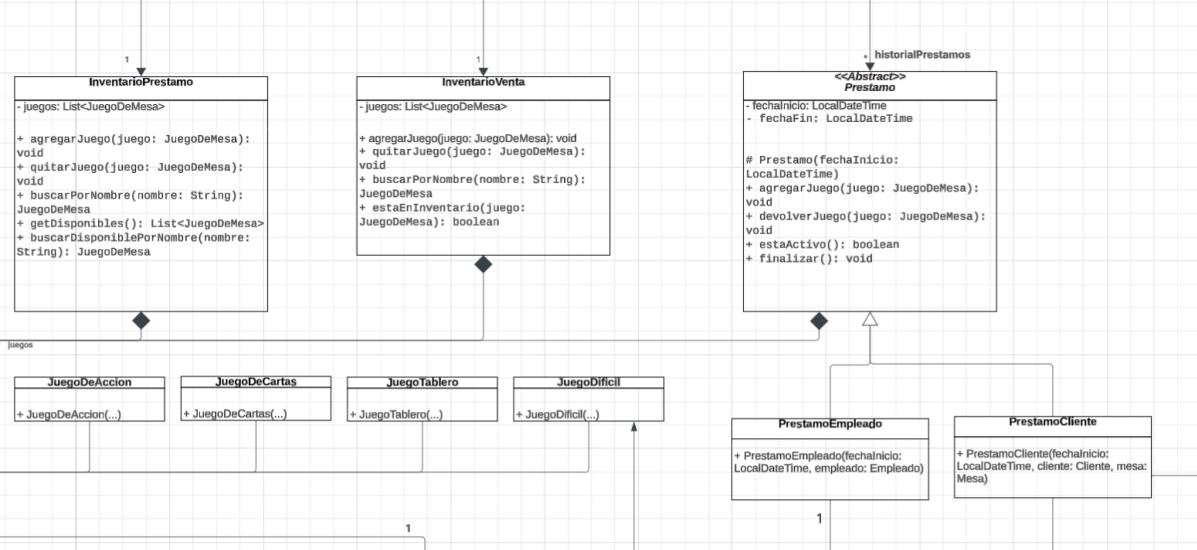
Salidas: Se genera la venta con mensajes mostrando lo utilizado o suelta error si se agregó mal la información o es inconsistente.

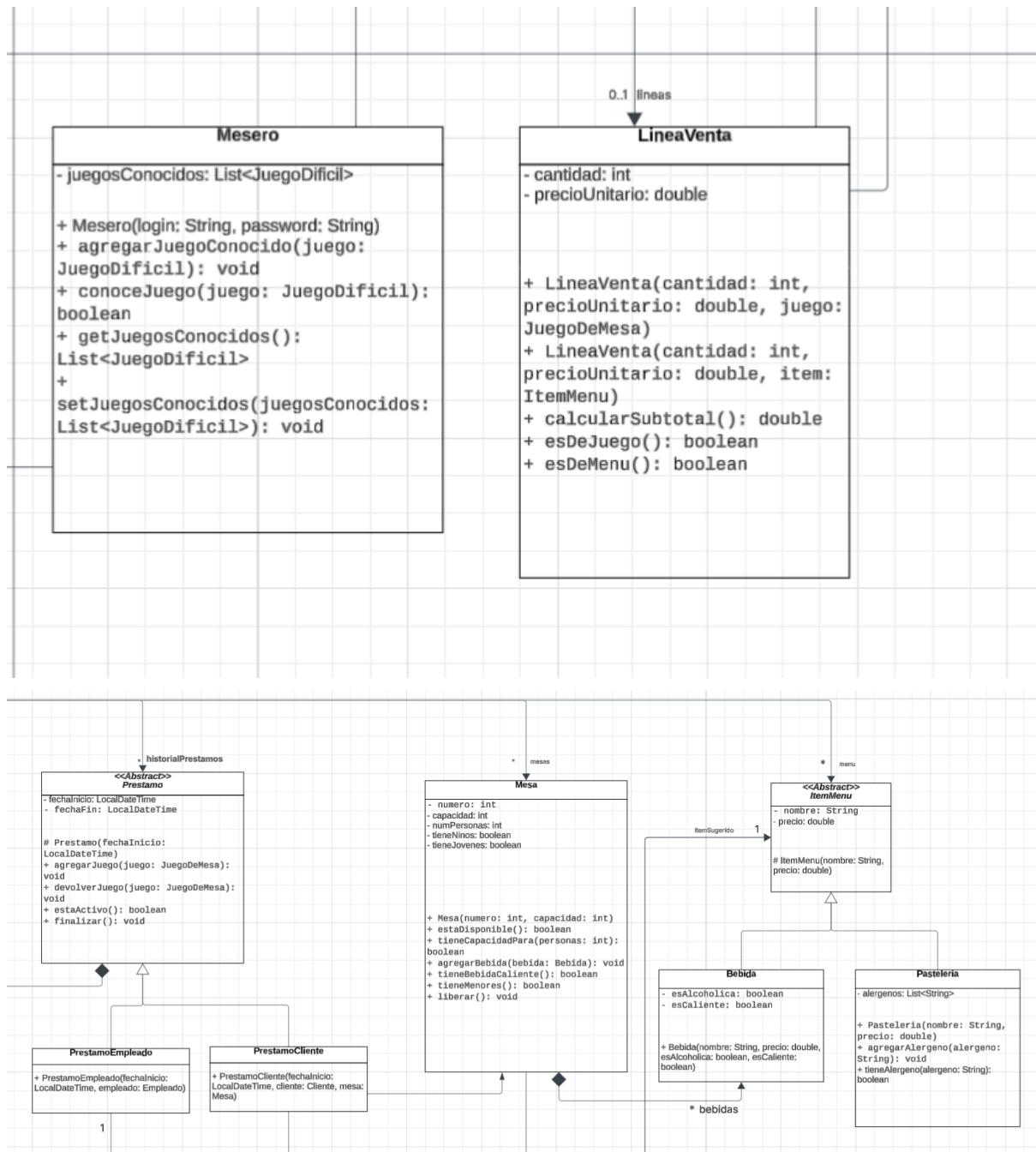
## UML-PROYECTO 2











[https://lucid.app/lucidchart/5cea1bd0-c58f-4e42-bc07-bca44bc7c342/edit?viewport\\_loc=-3741%2C-3750%2C16804%2C9541%2C0&invitationId=inv\\_3021c4fe-0d19-4394-b419-4831d0b363bb](https://lucid.app/lucidchart/5cea1bd0-c58f-4e42-bc07-bca44bc7c342/edit?viewport_loc=-3741%2C-3750%2C16804%2C9541%2C0&invitationId=inv_3021c4fe-0d19-4394-b419-4831d0b363bb)

En la imagen mostrada se encuentra el UML correspondiente al proyecto en la parte dos del proyecto. Además, se encuentra el link y por partes para su mejor comprensión.

## TORNEOS

Los torneos son una dinámica hecha por el Café con el fin de que los clientes compitan entre ellos activamente por diversión o una recompensa e incentivar las ventas dentro del local para mayor flujo de clientes. Estos se dividen en dos: los Amistosos y los Competitivos. Estos torneos cuentan con un juego y cupos disponibles; para estos cupos los usuarios pueden tener máximo tres. Si es de carácter amistoso la recompensa del ganador sería un bono de descuento, en cambio, si es competitivo el ganador puede llegar a ganar una suma monetaria y para su participación es necesario pagar una inscripción.

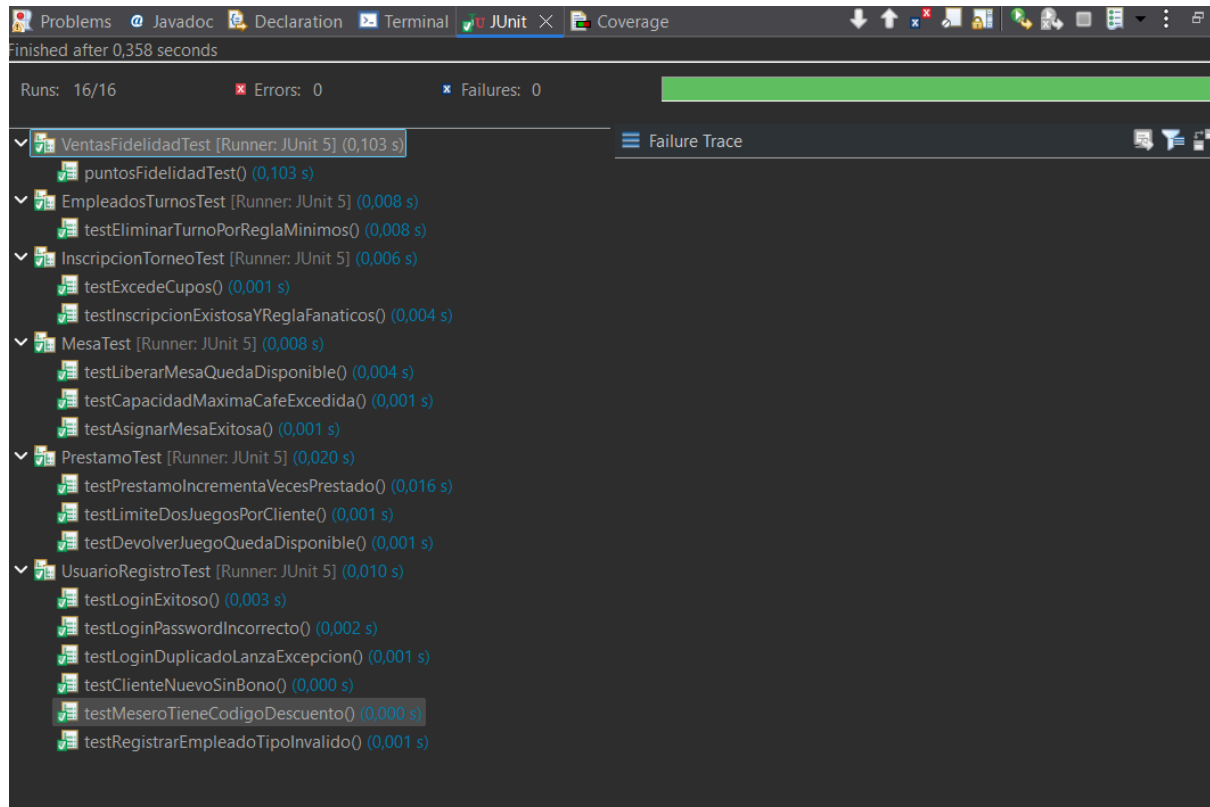
## LÓGICA - TESTS

Por la parte de la lógica, esta se divide en varias secciones dependiendo la funcionalidad de los métodos dentro de este.

1. Usuarios: Esta sección consiste en todo lo relacionado con los usuarios. Se encuentra el registro de las cuentas, ya sean para clientes, empleados o administradores. Asimismo, se tiene el método de inicio de sesión donde los usuarios acceden a sus cuentas.
2. Mesas: En esta parte se encuentran los métodos encargados de asignar y liberar las mesas de los usuarios. Además, está el de búsqueda de la mesa y suma de mesa dentro del Café.
3. Préstamos: Los métodos encargados del préstamos son los cuales generan estos dependiendo el cliente o empleado. Además, de las encargadas de la devolución de juego y finalización de de los mismos. También, se encuentran en las que se puede acceder a la información de los préstamos.
4. Ventas Cafetería: En esta parte se encuentra todo lo relacionado con la venta de pastelería y bebidas. El método encargado es la realización del pedido donde se realiza el pago directamente. Además tiene en cuenta descuentos, bonos y puntos.
5. Ventas Juego: Similar a la sección anterior, en esta parte se maneja la parte de juegos a la venta. El pedido del juego es el método correspondiente donde se realiza la venta inmediatamente teniendo en cuenta descuentos, bonos y puntos.
6. Inventario: Esta sección tiene todo lo relacionado con los inventarios de préstamos. Desde la inclusión de juegos en ambos inventarios, hasta movimientos de juegos desde los de venta hacia los de préstamo. Además, contiene el método donde se puede verificar el estado del juego, de este se puede ver si el juego necesita una reparación por sus daños. También, está la verificación de juegos que hayan sido robados o de el historial de préstamos.
7. Turnos: En la parte de turnos se encuentra la organización del horario de los trabajadores, donde estos pueden mirar el plan semanal de estos y mandar sugerencias al administrador acerca de estos. Además, se encuentran la asignación y eliminación de los turnos.
8. Menu: Esta parte consiste en lo relacionado con los alimentos de pastelería y bebidas. En esta se encuentran solicitudes de los empleados para la implementación de uno de estos ítems, la aceptación o el rechazo de estos, así como poder agregar o remover uno.
9. Favoritos: En esta se encuentran los métodos acerca de los juegos favoritos de los usuarios. Donde se puede agregar o eliminar juegos.
10. Reportes: Esta sección consiste en la obtención de la información financiera del café. En esta se accede a las ventas, propinas, impuestos y costos de un tiempo dado
11. Torneos: Consiste en los métodos en la organización del torneo. Desde la creación del mismo hasta la administración de los usuarios participantes; pudiendo inscribirse o describirlos. Además, de la premiación al ganador del mismo.
12. Métodos privados: Estos métodos sirven de ayuda para otros ya mencionados previamente, Entre los métodos que se encuentran en esta sección son: otorgar puntos de fidelidad, filtrar ventas, si hay mesero disponible, entre otras.

Para probar parte de la lógica se tienen unas pruebas donde se verifica la correcta funcionalidad de ciertos métodos. Estas pruebas se dividen en tres partes: Ventas fidelidad, Empleados turnos, Inscripción a torneo, Mesa, Préstamo y Registro. En la primera mencionada se puede encontrar cómo se evalúa la venta de un juego dependiendo ciertas características dadas previamente, entonces dependiendo el resultado del método se verifica si coincide con lo esperado o no. En la parte de empleados se crea un plan semanal con los turnos de algunos empleados y se evalúan funciones correspondientes a estos como eliminar turnos. Continuando, tenemos la relacionada con el torneo, donde antes de cada prueba se crea un torneo con sus características y se evalúan funciones como la

inscripción de torneo verificando que no exceda los cupos o los getters y setters. Ahora en la parte de tests para mesa se evalúa el método asignar método con los escenarios de: operación éxitos, capacidad máxima excedida y liberación de mesa libre. La prueba encargada de los préstamos se evalúa la solicitud del mismo, mirando las veces que incrementa, devolución del juego prestado y el límite de los juegos prestados. Para finalizar, se tiene la prueba del registro de usuario se mira cuando la contraseña es la incorrecta, cuando existen dos usuarios con el mismo login, se crea cliente nuevo sin bono y se registra un empleado el cual el tipo no es el válido.



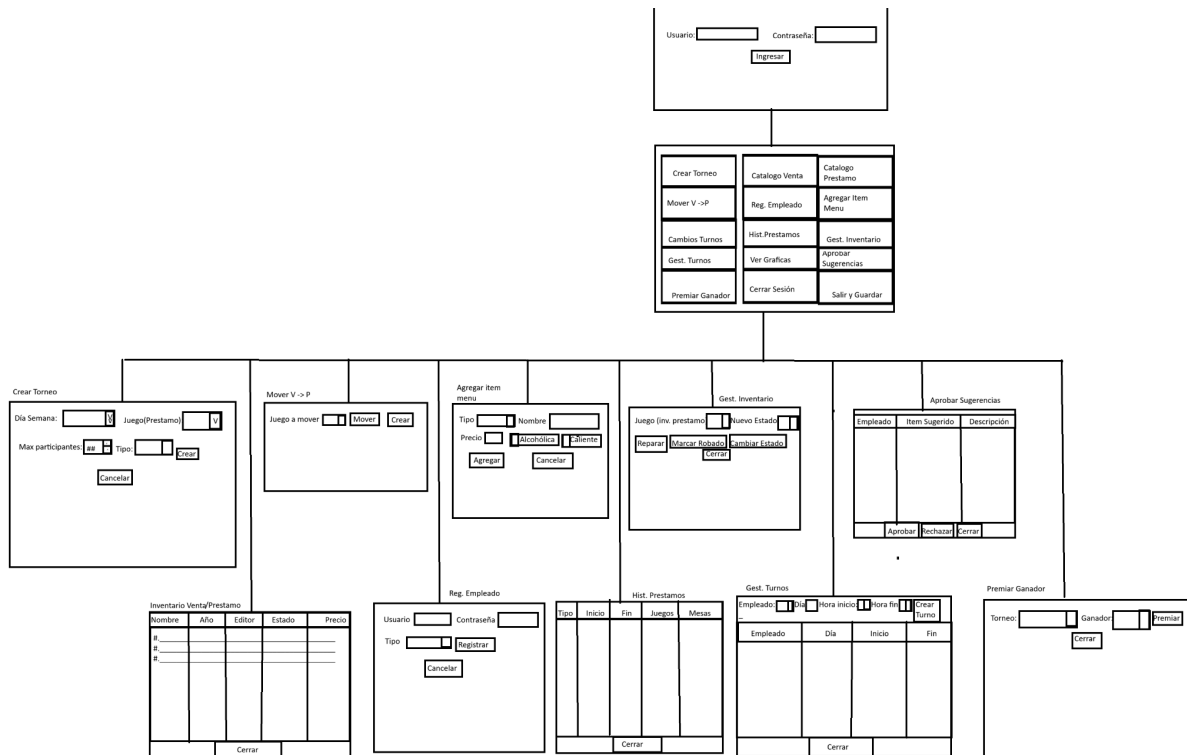
## MAIN

Por parte de la consola, esta se divide en dos partes. Primero tenemos la que es asignada para el administrador y la que está dirigida para los clientes. En la consola del administrador al inicializarla se empieza verificando si ya existen datos guardados se cargan, en cambio, si no se encuentran los archivos se crea un administrador nuevo con dos juegos, uno dirigido al inventario de préstamo y otro a la venta. Al finalizar la primera parte se revisa si se está registrado, si no lo está, se crea una cuenta de tipo administrador. A continuación, se despliega el menú principal; en este se encuentran cinco opciones. Las opciones mostradas son las siguientes: Crear Torneo, ver catálogo de ventas, ver catálogo de préstamos, desplazar juegos de ventas a préstamos y la opción de salir y guardar. En la opción de creación del torneo, se le pide al administrador la información relacionada con el torneo, entonces se evalúa si estas son correctas para la creación del mismo, sino se crea un error explicando el porqué. En las opciones de mostrar el catálogo se muestra directamente el mismo si es que no está vacío. En la opción de movimiento de elementos entre inventarios primero muestra los juegos disponibles si es que no está vacío, para posteriormente pedirle al administrador el nombre del juego para intercambiarlo y al final le dice si se movió o si hubo un error a la hora de digitar el juego.

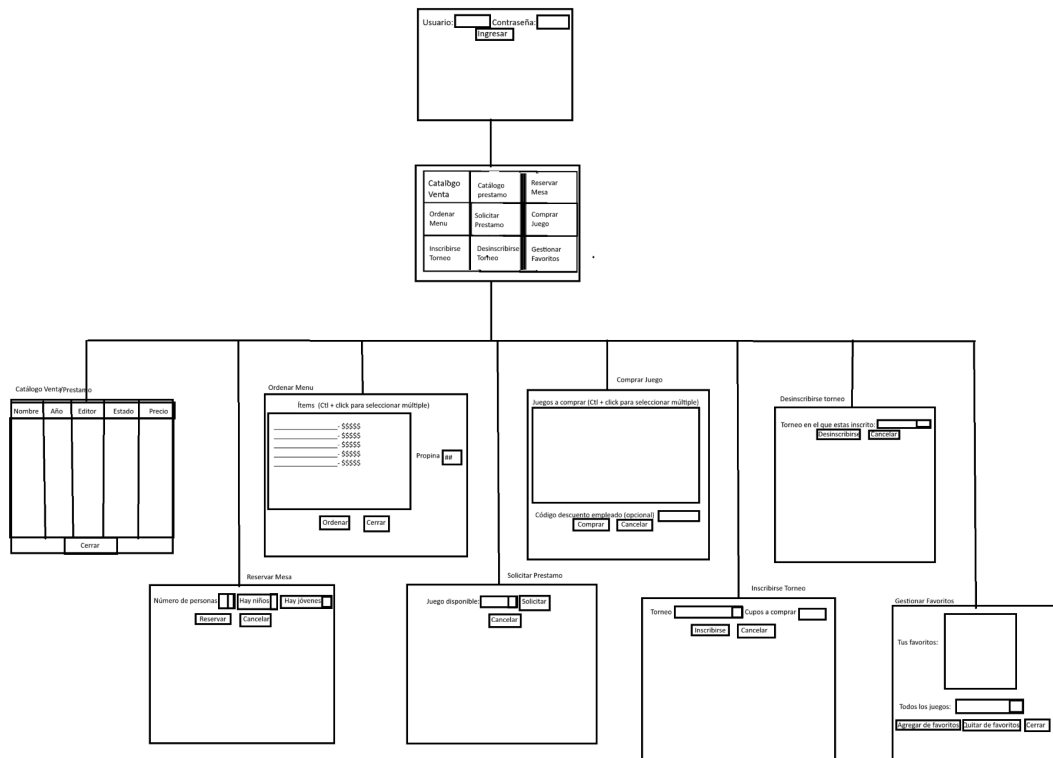
Por parte de la consola del cliente, este inicia intentando cargar los datos que ya están guardados. Sin embargo, si no se encuentran los archivos se inicializa desde cero, todo en blanco. Después, le da la bienvenida al cliente para verificar si ya está logueado le muestra las opciones de registrarse o iniciar sesión. En ambas pide usuario y contraseña; si el inicio de sesión fue un éxito, sigue con los siguientes pasos, en cambio si no se inició sesión correctamente suelta un error. Por parte del registro le pide al cliente la información de la contraseña y el nombre de usuario, finalizando le dice si el registro fue un éxito o si hubo un error. Posteriormente, le da oficialmente la bienvenida al menú con sus correspondientes opciones. Las opciones son: Ver los catálogos, Inscribirse a torneos, Compra de juego y la opción de salir guardando los datos. Después dirige al cliente a la opción seleccionada. En la opción de vistas de catálogos muestra por separado la parte de préstamo y la de venta. En la opción de inscribirse en un torneo, primero le muestra todos los torneos disponibles de acuerdo al día, tipo y cupos libres. Después, le pide el nombre del juego el cual el cliente está interesado, lanza error si no se encuentra el juego, sino pregunta el número de cupos a reservar. Finaliza diciendo que el registro fue un éxito o dice si hubo un error en la inscripción. En la opción de compra de un juego, primero se ingresa el juego a comprar, después de filtrar el juego le pregunta si se tiene descuento o bono, además si se van a usar los puntos de fidelidad. Al finalizar verifica la información suministrada por el cliente y arroja error si ingresa algún dato mal, sin embargo, si la compra fue un éxito le muestra el precio incluyendo IVA y descuento con los puntos adquiridos por la compra. Por último, está la opción de salir y guardar, la cual consiste en exportar los datos a la carpeta correspondiente y finaliza el programa.

## **INTERFAZ GRÁFICA**

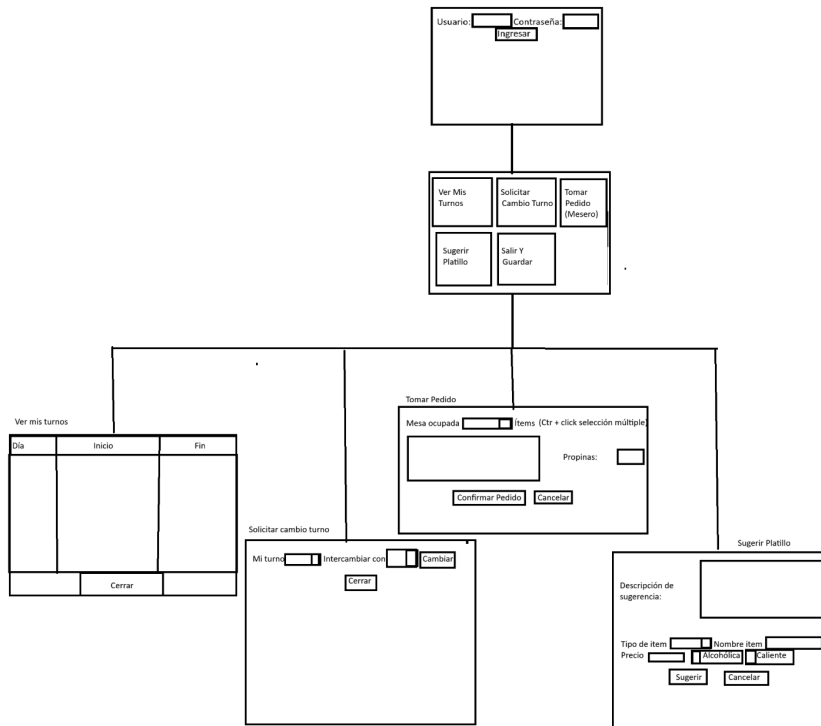
Para la parte de interfaz gráfica, antes de empezar a programar todo lo relacionado con la aplicación, se decidió tener un boceto inicial de cómo se espera que se vea esta para los distintos usuarios, ya sea administrador, cliente o empleado. Para esto la planeación y el modelo se dividió en tres partes, las cuales son los distintos tipos de usuarios. Además, se puede decir que el diseño de toda la interfaz gráfica se hizo para que al mismo usuario no le sea difícil de entender su funcionamiento. A continuación se mostrará como se verá la interfaz gráfica.



La imagen mostrada es la correspondiente a la interfaz del administrador. Lo primero que se debe hacer es el ingreso de sesión, para poder ingresar a la cuenta y poder ver el menú de acciones. En este menú se encuentran diversas acciones las cuales dirigen a distintas ventanas con sus correspondientes funciones. Por ejemplo, se pueden ver ambos tipos de inventarios, ya sea de venta o de préstamo, como una lista de los juegos disponibles. Además, de la opción de poder mover los juegos entre los inventarios. En cuanto a la gestión de los empleados se puede encontrar que este mismo los puede registrar con sus características, también puede estar atento a sus turnos y sugerencias. Puede encargarse de sus deberes acerca de los torneos, pudiendo crearlos y seleccionando al ganador del mismo. Con todo lo ya mencionado se puede decir que este modelo de interfaz cubre todas las necesidades del administrador.



Anteriormente se mostró la imagen correspondiente a la interfaz gráfica de los usuarios, se puede ver que esta es menos compleja comparada con la del administrador sin perder las necesidades del mismo cliente. Desde la vista de los catálogos hasta la participación de los torneos, este puede interactuar para realizar ya sean pedidos de alimentos, inscripción a los torneos hasta pedidos de los juegos dependiendo lo que espera el cliente. Las nueve opciones mostradas son capaces de satisfacer al cliente.



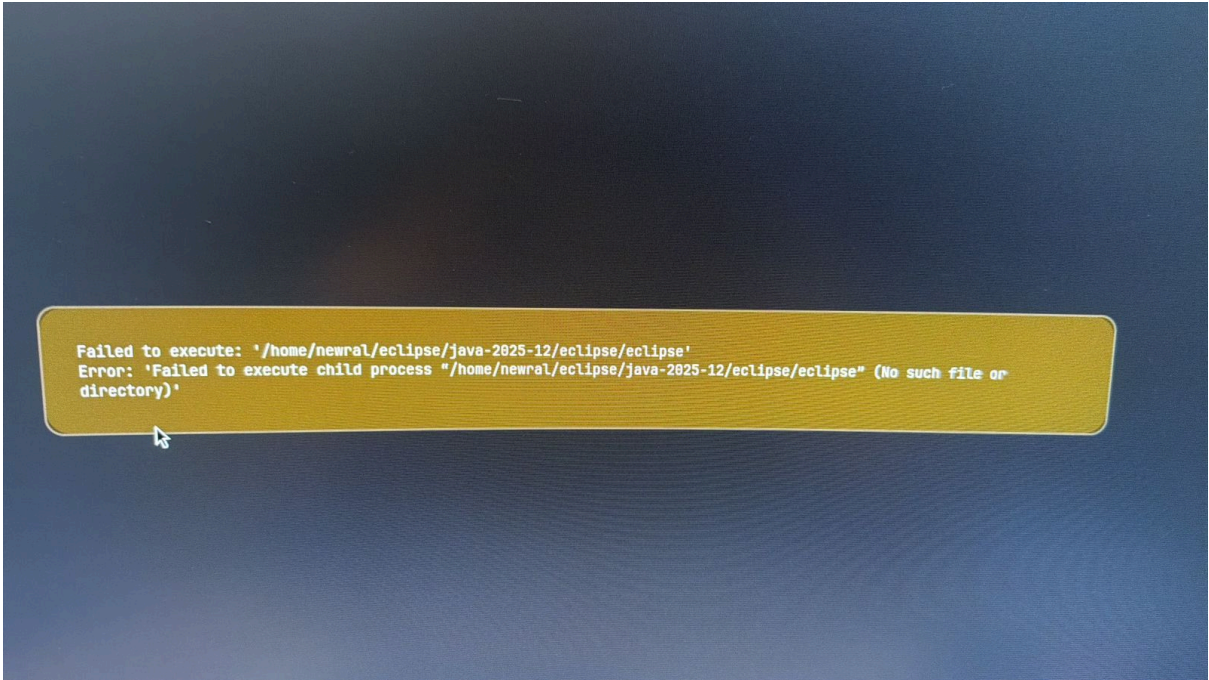
Por último, tenemos la interfaz que va dirigida a los empleados en general, ya sean cocineros y meseros. La diferencia principal de estos dos solo radica en la opción de tomar el pedido del cliente, debido a que es exclusivamente del mesero. En cambio, el resto, ya sea ver los turnos, solicitar cambio de turno o sugerir platillo al administrador. También, se puede decir que este cumple con todas las necesidades de los empleados.

## USO DE IAG

Usamos IA para automatizar los getters y setters a la hora de desarrollar las clases de las entidades presentes en el paquete de modelo.

A su vez, la usamos para hacer el código más compacto en términos de líneas de código ya que la implementación fue más larga de lo esperada. Con esto buscamos tener un código más legible, más navegable y más claro al momento de entender. La IA, básicamente, nos recomienda hacer los métodos en una línea respetando los corchetes al momento de definir su contenido.

Antes de exponer las cosas puntuales con las cuales nos ayudó, decir que yo, Nicolas Arce, tuve un problema al momento de cambiar de la rama main a mi propia rama después de avanzar con la lógica. Uso lazy git, por lo que al ubicarme en el cuadro de las ramas y darle space a la mía, me pedía hacer un stash y luego un pop. Le di enter (ya que lo hace automáticamente) y, además de sobrescribir cosas sobre el código que venía desarrollando, me detono eclipse. Fuera de los errores de sintaxis por las sobreescrituras, me salían errores generales en el proyecto con un signo de exclamación rojo. Entendiendo que no era sintaxis, intenté hacerle track en problems pero de verdad que no encontré de donde venía el problema. Decidí reiniciar el IDE y cuando lo intente abrir de nuevo, me aparece esto:

A screenshot of a terminal window with a dark blue background. A yellow rectangular box highlights an error message. The message is in white text and reads: "Failed to execute: '/home/newral/eclipse/java-2025-12/eclipse/eclipse'" followed by "Error: 'Failed to execute child process "/home/newral/eclipse/java-2025-12/eclipse/eclipse" (No such file or directory)'". A white mouse cursor is pointing at the bottom left of the yellow box.

```
Failed to execute: '/home/newral/eclipse/java-2025-12/eclipse/eclipse'
Error: 'Failed to execute child process "/home/newral/eclipse/java-2025-12/eclipse/eclipse" (No such file or
directory)'
```

Por lo que como último recurso llamé a Claude Code al proyecto y, ya habiendo explicado la situación, le di un prompt para que me ayudara a solucionar el problema (ya que quizás era algo que Git modificó y yo genuinamente no me había dado cuenta). Lo intentó modificar pero sigue sin abrirme el IDE (la verdad es un problema).

Fuera de esta situación, usamos la IA para entender mejor cómo podemos manipular el tema de las fechas. Nos recomendó `LocalDate` y `LocalDateTime`, por lo que a partir de eso leímos su documentación respectiva.

Por último, nos recomendó usar `System.out.println` para aquellos casos donde no haya una excepción, pero al usuario se le deba indicar una información necesaria como los alérgenos en su comida como tip.